

Enigma3 Architecture

2026-09-16 · @Someone

Vision

Enigma3 is Enigma2 re-architected as a small set of cooperating processes instead of one monolithic binary, so a Broadcom STB's multiple ARM cores and GPU each carry dedicated work instead of racing through a single thread of control.

Same skins, same plugin API, same DVB stack — split across process boundaries so a crash in the UI, a plugin, or the GPU driver never blacks out the screen or drops an in-progress recording.

Motivation

Enigma2 is architecturally a single process, single address space: one `eMainloop` (`lib/base/ebase.h`) funnels tuner events, timers, GUI redraws and Python callbacks through one reactor. `Navigation` (`lib/nav/core.cpp`) ties service playback directly into the same process as the GUI. `lib/dvb` , `lib/gdi` and the Python plugin runtime all live in one binary sharing one thread of control for anything not explicitly backgrounded.

Problem today	Consequence
One address space for GUI, plugins, DVB and rendering	A plugin bug or Python GC pause can stall tuning, recording, and AV sync
No expressed core affinity	The OS scheduler, not the architecture, decides what runs where — you can't pin graphics to one core and UI/plugins to another
<code>gRC</code> 's render thread has no scheduling policy	No frame-pacing guarantee under load (fast EPG scroll, subtitle bursts)
One crash takes down everything	A segfault anywhere is a black screen and a dropped recording
Closed-source Broadcom EGL/GLES driver runs in-process	A driver crash is indistinguishable from an application crash

Enigma3 keeps the same hardware-facing code (`lib/dvb` , `lib/gdi`) and the same Python plugin API, and reorganizes the process boundaries around them.

Target hardware model

Broadcom STB SoCs used across this ecosystem (BCM72xx-BCM74xx families) give you, on one chip: 2-4 ARM cores, a GPU capable of EGL/GLES3, hardware demux with a fixed number of PID-filter slots, and dedicated hardware AV decode/sync blocks with their own display-plane compositor (video plane + graphics/OSD plane blended in hardware, independent of the GPU).

"Exploiting multi-core" here does not mean spreading work across as many processes as possible — it means giving latency-critical work (tuning, AV sync, GPU compositing) dedicated, schedulable contexts instead of serializing everything through Enigma2's one mainloop, while hardware continues to do AV decode/demux exactly as it does today.

Reference target: Dreambox DM900 UHD 4K — Broadcom BCM7252S, dual-core Cortex-A15. On a 2-core SoC the payoff is concrete and small: general/UI/Python work pinned to one core, GPU submission/render pinned to the other, each in its own address space, so a crash or stall in one never propagates to the other.

System architecture overview

Five to six long-lived processes, each with a clear ownership boundary, supervised by a minimal init. `e3-tuner` and `e3-avd` share a core (chatty PCR/PID/CA traffic between them) but stay separate processes so a demux/scan hang is recoverable without dropping a decode in progress. `e3-comp` is isolated on its own core because it is the one process with hard real-time obligations (vsync, frame pacing). `e3-ui` and `e3-pluginhost` share a core deliberately — both are soft, non-frame-critical workloads, and this is where the RAM/CPU cost of splitting anything at all gets clawed back.

flowchart TD

```

SUP["e3-supervisord<br/>process mgmt, core affinity, watchdog"]
SUP --- BUS[("control bus-typed IPC over unix sockets")]
BUS --- TUNER["e3-tuner<br/>demux, CA, EPG scan"]
BUS --- AV["e3-avd<br/>decode/AV sync/trickplay"]
BUS --- COMP["e3-comp<br/>EGL/GLES3 compositor, OSD plane"]
BUS --- UI["e3-ui<br/>skins, widgets, input"]
BUS --- PLUGIN["e3-pluginhost<br/>sandboxed Python plugins"]
TUNER === AV
UI -.shared-memory data plane.- COMP
AV -.frame metadata.- COMP

```

Video itself never crosses the data plane: hardware demux → hardware decoder → hardware video plane, exactly as today. `e3-comp` only owns the graphics/OSD plane and blends against the video plane at the hardware-compositor level.

Process specifications

Process	Core (DM900, 2-core example)	Responsibilities	Crash-restart policy
e3-supervisord	core 0, low priority	Process lifecycle, declared core-affinity/priority manifest, heartbeat watchdog	N/A — the one process that must not crash; kept deliberately minimal
e3-tuner	core 1	Tuner/demux ownership, hardware PID filters, CA, EPG scanning	Restart drops the current scan/CA session; recording in e3-avd should survive if the handoff is already established
e3-avd (+ e3-record)	core 1	AV pipeline management, PCR/AV sync, trickplay, PVR/timeshift writing	Restart mid-recording is the hardest case to get right — the primary reason for the split in the first place
e3-comp	core 1 (dedicated on ≥3-core SoCs)	EGL/GLES3 compositor, OSD plane, hardware-plane blending	Restart reattaches e3-ui's live scene graph; UI blinks, tuning/recording unaffected
e3-ui	core 0	Skins, widgets, input, scene-graph submission to e3-comp	Restart is a one-second UI blink, no effect on tuner/AV
e3-pluginhost	core 0 (shares with e3-ui)	Sandboxed Python plugin execution behind a compatibility shim	Restart isolates a misbehaving plugin from the rest of the system
e3-netd	core 0	HbbTV/OTT streaming, mDNS/avahi, remote API (OpenWebif-equivalent)	Independent of the AV/UI split

On a quad-core SoC, e3-tuner / e3-avd and e3-comp each get their own core, leaving e3-ui / e3-pluginhost / e3-netd / e3-supervisord to share the fourth — see Core affinity and scheduling below.

IPC design

Two planes, not one:

- **Control plane** — low-frequency, latency-tolerant: channel change, timer events, EPG queries, plugin lifecycle, CA status. A typed RPC layer (schema-driven, generating both C++ and Python stubs) over `AF_UNIX` sockets, replacing today's ad hoc SWIG surface with something versionable.
- **Data plane** — high-frequency, latency-sensitive: OSD scene-graph updates, subtitle bitmaps, frame/PCR stats, EPG deltas. Lock-free shared-memory ring buffers, avoiding both copies and context-switch-per-message overhead.

Milestone 1 (see Milestone 1 status below) builds the first real instance of the data plane: a shared-memory ring (`gShmOpcodeRing`) carrying `gRC` 's existing opcode stream (`gOpcode` , `lib/gdi/grc.h`) between the UI process and the compositor process, POSIX `shm_open` + `mmap` with a `PTHREAD_PROCESS_SHARED` mutex/cond, plus a companion byte arena for variable-length payload (text, region rect lists, gradient colors, inline pixel bytes). This reuses the existing, already-async opcode-queue abstraction instead of inventing a new scene-graph format — `gRC` already serializes every draw call into a queue consumed by a dedicated thread; the milestone's job is to move that queue across a process boundary, not redesign it.

Video frames and large pixmaps are the one payload type this plane is not meant to carry as raw bytes at scale: an accel/ION-backed pixmap should hand over its dma-buf fd (already shareable via `ION_IOC_SHARE`) rather than copying pixel data, reusing the zero-copy GPU-import path (`gtexture_manager.cpp` 's `createTextureFromDmabuf()`) that already exists in this codebase for exactly this purpose.

Object-oriented core library

`libenigma3-core` , shared by every process, modernizes `lib/base` rather than replacing it wholesale:

Interface	Replaces	Notes
<code>IEventLoop</code>	<code>eMainloop</code> (<code>lib/base/ebase.h</code>)	Same reactor role, but each process gets its own instance instead of one shared global loop; async I/O expressed via C++20 coroutines where it clarifies control flow (tuner scan sequencing), not as a blanket rewrite

Interface	Replaces	Notes
<code>Signal<Args...></code>	<code>eSignal</code> /SWIG-glued callbacks	Type-safe, usable identically in-process and (via the RPC layer) across processes
<code>IDemux</code> , <code>IDecoder</code> , <code>ITuner</code>	Direct <code>lib/dvb</code> calls	Implemented today by Broadcom-backed classes wrapping the existing <code>lib/dvb</code> code unchanged — not a rewrite of the hardware talk, an interface drawn around it
<code>ICompositor</code>	Direct <code>lib/gdi</code> calls	Implemented today by a class wrapping <code>gEGLDC</code> / <code>gLCDDC</code> (<code>lib/gdi/egl</code> , <code>lib/gdi/glcddc.*</code>)

Every hardware handle (DVB device fds, EGL contexts, decoder handles) gets an RAII wrapper, so a process crash/restart never leaks a PID-filter slot or a decoder instance the way a leaked raw fd can today.

Core affinity and scheduling

SoC	Core 0	Core 1	Core 2	Core 3
BCM7252S (DM900, dual-core)	<code>e3-ui</code> , <code>e3-pluginhost</code> , <code>e3-netd</code> , <code>e3-supervisord</code>	<code>e3-tuner</code> , <code>e3-avd</code> , <code>e3-comp</code> (all three, unpinned within the core)	—	—
Quad-core (e.g. BCM7278-class)	<code>e3-supervisord</code> , <code>e3-netd</code>	<code>e3-tuner</code> , <code>e3-avd</code>	<code>e3-comp</code> (dedicated, <code>SCHED_FIFO</code> at modest priority for frame pacing)	<code>e3-ui</code> , <code>e3-pluginhost</code>

`e3-comp` 's render thread gets explicit `pthread_setaffinity_np` pinning and, where it has a dedicated core, a real-time scheduling class — net-new versus today's `gRC::thread()` , which has neither affinity nor scheduling policy set at all (confirmed by reading `lib/gdi/grc.cpp` : the render thread runs at default priority with no `sched_setscheduler` / `pthread_setname_np` anywhere in `lib/gdi`).

On the 2-core DM900 case, the meaningful win is not spreading five processes across five cores — it's precisely the UI/plugins-vs-render split: GC pauses, plugin stalls and EPG parsing on core 0 no longer contend with GPU submission on core 1.

Fault isolation and supervision

`e3-supervisord` is deliberately dumb, not a general init system: it starts processes with a declared core-affinity/priority/restart-policy manifest, watches heartbeats over the control bus, and restarts on failure per policy.

- `e3-ui` restarts silently and reattaches to `e3-comp`'s live scene graph — a one-second blink, nothing else affected.
- `e3-tuner` / `e3-avd` restarting mid-recording is the hardest case, and the one that matters most: it's the actual payoff for the added complexity.
- `e3-comp` restarting is detected by the UI process via a dropped shared-memory connection (socket EOF / heartbeat timeout); it respawns, reconnects, and lazily resends glyph/texture cache entries on the next draw that misses — no explicit resync protocol needed, mirroring how the in-process texture/glyph atlas cache already behaves today.

Today, a segfault anywhere is a black screen and a dropped recording. Under this design, most crashes are a UI blink.

Migration path

Not a rewrite of `lib/dvb` / `lib/gdi` — wrap them, one seam at a time, each step independently shippable:

1. Extract today's Navigation/service/DVB code into `e3-tuner` + `e3-avd` behind the new `IDemux` / `IDecoder` interfaces, keeping the actual Broadcom driver calls unchanged.
2. Split the compositor out next — turning `gEGLDC` / `gRC` into a standalone process fed by an IPC scene graph. **This is Milestone 1, and it's underway; see the status below.**
3. Move Python/plugins into `e3-pluginhost`, with a compatibility shim so existing third-party plugins keep working against the old API surface, translated to the new RPC underneath.
4. Split the UI process last, once the scene-graph protocol to `e3-comp` is proven.

Before committing further past Milestone 1, measure the RSS delta on real, lowest-spec target hardware — five processes each with their own runtime (especially a full Python interpreter in `e3-pluginhost`) costs real memory on a 512MB box, and that number

should decide whether `e3-ui / e3-pluginhost` need to merge into one process to fit.

Milestone 1 status

Written against this repository's `enigma3-compositor-split` branch (off `gles-egl`). Nothing here has been compiled or run — no toolchain was available while writing it; it needs verification on a real cross-toolchain and DM900 hardware before it's trusted.

Component	File(s)	Status
Wire opcode format	<code>lib/gdi/ipc/gwireopcode.h</code>	Done — flat POD struct, not a union, to avoid shared-memory placement-new landmines
Shared-memory transport	<code>lib/gdi/ipc/gshmopcodering.h</code> , <code>cpp</code>	Done — SPSC ring + byte arena, aligned payload allocation (a real alignment bug was found and fixed during review)
Opcode serialization (UI to wire)	<code>lib/gdi/ipc/gopcodeserialize.h</code> , <code>cpp</code>	Done for fill/blit(non-accel)/rectangle/line/text(no out-param)/clip/colors/gradients/borders/offsets/flush/flip/vsync/spinner/show/hide/shutdown
gRC producer-side integration	<code>lib/gdi/grc.h</code> , <code>cpp</code>	Done — <code>connectToCompositor()</code> plus a forwarding branch in <code>submit()</code> , with cleanup mirrored case-by-case against the real <code>gDC::exec()</code> . Entirely inert: gated behind <code>HAVE_E3_COMPD_CLIENT</code> , defined by no build file
UI-process proxy <code>gMainDC</code>	<code>lib/gdi/ipc/gcompositorproxydc.h</code> , <code>cpp</code> , <code>lib/gdi/gmaindc.h</code>	Done — satisfies the <code>gMainDC</code> singleton when there's no local <code>gEGLDC</code> ; <code>needsSpinnerSetup()</code> lets <code>main/enigma.cpp</code> skip local spinner setup
Compositor process	<code>main/e3-compd.cpp</code>	Done — real <code>gEGLDC</code> construction plus a full opcode deserializer, cross-checked against <code>gegldc.cpp</code> 's actual <code>exec()</code> switch; falls back to opcode logging when built without <code>HAVE_EGL / HAVE_DREAMBOX_EGL</code>
Build wiring	<code>lib/gdi/Makefile.inc</code> , <code>main/Makefile.am</code>	Done — new <code>libenigma_gdi_ipc.a</code> so <code>e3-compd</code> doesn't need Python/DVB dependencies just for the transport; <code>e3-compd</code> added to <code>bin_PROGRAMS</code>

Not yet done: `renderPara / glyph` text (needs `eTextPara` flattened to a stable-keyed glyph run); accel/ION-backed blits (needs dma-buf fd handoff over the control socket instead of inline bytes); a resolution handshake (the proxy DC's `size()` is unusable until something calls `setResolution()`); any process supervisor or reconnect logic; per-pixmap texture caching across the wire (every blit currently re-uploads).

Risks and tradeoffs

Risk	Detail	Mitigation
Memory budget	Many STBs in this ecosystem have 512MB-1GB RAM; five processes, each with runtime overhead (a full Python interpreter in <code>e3-pluginhost</code> especially), costs real memory	Measure RSS delta after Milestone 1 (compositor split alone) before committing to the full topology; merge <code>e3-ui</code> / <code>e3-pluginhost</code> if the lowest-spec target doesn't fit
IPC latency vs. monolithic calls	Every cross-process opcode/RPC is strictly slower than an in-process function call	The data plane (shared-memory rings) is specifically designed to minimize this; measure opcode-submit-to-pixel-on-screen latency against the single-process baseline
Complexity cost	More moving parts, more failure modes to reason about, a real supervisor to build and get right	Incremental rollout (see Migration path) — each milestone is independently shippable and reversible
Unverified build	Milestone 1's code has not been compiled; no cross-toolchain or DM900 hardware was available while writing it	Build and smoke-test on the real toolchain before relying on any of it; several specific gaps are flagged in Milestone 1 status
Mutual exclusion not enforced at configure time	A build could theoretically enable both a real <code>gMainDC</code> backend and the compositor-proxy path simultaneously	Currently fails loudly via <code>gMainDC</code> 's singleton assert at startup (safe failure, not silent misbehavior) rather than being prevented at configure time — a real <code>configure.ac</code> option is future work

Roadmap

1. **Finish Milestone 1:** resolution handshake (`e3-compd` reports real display geometry back to `gCompositorProxyDC`), glyph-run flattening for `renderPara /text` (with `glyph_key` rekeyed off stable font-path/size/glyph-index identity instead of today's in-process FreeType pointer), dma-buf handoff for accel/ION-backed blits, and a minimal supervisor (detect a dropped `e3-compd` connection, respawn, reconnect).
2. **Milestone 2 — tuner/AV split:** extract `lib/dvb / Navigation` into `e3-tunerd / e3-avd` behind `IDemux / IDecoder` , with recording surviving an `e3-avd` restart as the acceptance test.
3. **Milestone 3 — plugin sandboxing:** move Python into `e3-pluginhost` with a compatibility shim, so existing third-party plugins keep working unmodified against the old API surface.
4. **Milestone 4 — UI process split:** the final piece, once the scene-graph protocol to `e3-comp` is proven under real navigation load (fast EPG scrolling, subtitle bursts).
5. **Generalize the control-plane bus:** replace the purpose-built opcode transport with the general typed-IDL bus (schema-driven RPC, C++ and Python stubs) described in the IPC design section, once the compositor case has proven the pattern on real hardware.